TapTap Blackbucks

BLACKBUCK
EDUCATION

Project Title

Automated Email Sender

Submitted By

Name	Ayushya Lakshmi Nagireddy
College Name	International school of technology and science
Department	B-Tech(CSE)
Academic Year	3 Year (1 Sem)

Submitted to

TapTap Blackbucks Product

Table of Contents

Introduction

In the modern world, email remains one of the most widely used and relied-upon forms of digital communication. From businesses sending invoices and updates to institutions sending reminders and notifications, email plays a central role in keeping people informed and connected. However, the process of manually composing and sending individual emails, especially to large groups on a recurring basis, is time-consuming, repetitive, and prone to mistakes.

The Automated Email Sender project was developed to address this exact problem. The system is a Python-based application that handles the entire process of composing, scheduling, and delivering emails without requiring a person to manually intervene each time. Once configured, it can send personalized emails to multiple recipients, attach relevant documents, and repeat this process automatically at scheduled intervals.

This project demonstrates how relatively simple Python libraries, when combined in a well-structured application, can replace a tedious manual task with a smooth, reliable, and fully automated workflow. It is particularly useful in academic institutions, small businesses, and organizations that regularly send reminders, reports, or bulk notifications.

Objectives

• To design and develop a Python-based system that automates the process of sending emails without manual intervention.

• To integrate SMTP (Simple Mail Transfer Protocol) for secure and reliable email

transmission.

• To support file attachments, allowing documents, PDFs, and images to be sent along with emails.

• To implement a scheduling mechanism that triggers email delivery at defined times or intervals.

• To manage bulk email communication by reading recipient data from a structured CSV dataset.

• To log every email send attempt so that success and failure records are maintained for reference.

• To reduce manual effort in repetitive communication tasks and improve overall workflow efficiency.

Problem statement

Sending repetitive emails manually is one of the most overlooked yet time-consuming tasks in both academic and professional environments. Consider a scenario where a teacher needs to send assignment reminders to 60 students every Monday, or a company needs to email monthly salary slips to all 200 employees. Doing this by hand means opening an email client, typing the message, attaching a file, entering each recipient's address, and hitting send, one by one. This not only consumes valuable time but also introduces the risk of human errors such as missing recipients, sending to the wrong person, forgetting an attachment, or simply forgetting to send the email at all.

There is a clear need for a lightweight, programmable, and easy-to-configure system that can take over this task completely. The Automated Email Sender fills this gap by providing a reliable, code-driven solution that handles everything from composing the message to delivering it, on schedule, every time.

Algorithm

Step 1: Start the application.
Step 2: Load the sender's email credentials, SMTP server details, and port number.
Step 3: Read the recipient information from the recipients.csv file.
Step 4: Validate the recipient details such as name, email address, attachment path, image path, and scheduled time if applicable.
Step 5: Create the email by setting the sender, recipient, subject, and message body.
Step 6: Attach the required files (PDF, documents, etc.) if an attachment is specified.
Step 7: Embed the image in the email body if an image is provided.
Step 8: Establish a secure connection with the SMTP server using TLS.
Step 9: Authenticate the sender using the configured email credentials.
Step 10: If scheduling is enabled, wait until the scheduled time specified in the CSV file.
Step 11: Send the email to the corresponding recipient.
Step 12: Record the email delivery status (Success/Failure) in the log file.
Step 13: Repeat Steps 5–12 for all recipients in the CSV file.
Step 14: Close the SMTP connection after all emails have been processed.
Step 15: Display the summary of the email sending process.
Step 16: End the application.

Conclusion

The Automated Email Sender project successfully automates the process of sending emails to one or multiple recipients, reducing the time and effort required for manual email communication. The system reads recipient information from a CSV file, composes personalized emails, supports file attachments and inline images, and sends emails securely using the SMTP protocol. It also provides scheduling functionality, allowing emails to be delivered automatically at the specified time.

The project improves communication efficiency by minimizing human errors, ensuring reliable email delivery, and simplifying repetitive email tasks. The logging feature helps monitor email delivery status, making the system more reliable and easier to manage. Overall, the Automated Email Sender is a simple, efficient, and scalable solution for automating email communication and can be further enhanced with features such as a graphical user interface, database integration, and advanced email templates in future versions.